American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Mid-Term Project Report

Spring 2025-2026

Section: B

Group: 4

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: Rafiya Tasnim ID: 22-49284-3	Load and inspect dataset Missing values handle Outliers handle Report Writting
2.	Name:Ishrak Shahnawaz ID:22-49286-3	Duplicate rows Convert data Normalization
3.	Name:Md. Sifti Al Mahmud ID:23-50099-1	Handle invalid data Filter data Balance dataset Train/Test split
4.	Name:Rajbir Acharjee ID:22-48080-2	Descriptive statistics Compare averages Compare variability

Dataset Description

This dataset contains 5000 records of climate related data across 16 variables for various countries in 2026. It includes many environment indicators like average temperature, temperature change, CO_2 emissions and sea level rise. It also covers extreme weather events like heatwaves, wildfires, floods and droughts. Additional metrics include rainfall changes, air quality, climate risk scores and population affected. The data is useful for analyzing climate patterns, risk assessment and environmental impact in different regions of the world.

Project Implementation Detail

Load the Dataset:

The dataset exists as a CSV file on computer but R cannot directly access it. So the dataset neede to be loaded in R.

```
data <- read.csv("data_science_project.csv", stringsAsFactors = FALSE)

head(data)
dim(data)
```

```
> head(data)
  record_id        country        region avg_temperature_c temperature_change_c
1   CLM0001         Canada North America              16.8                 2.40
2   CLM0002         Brazil South America              28.8                 1.51
3   CLM0003      Australia       Oceania               6.6                 1.70
4   CLM0004        Germany        Europe              10.9                 1.53
5   CLM0005  United States North America              13.9                 2.07
6   CLM0006 United Kingdom        Europe               6.2                 0.64
  co2_emissions_mt sea_level_rise_mm flood_risk drought_risk heatwave_days
1             5477              1.44        Low         High             9
2             3758              1.55     Medium       Medium            38
3             1904              5.81     Medium       Medium            20
4             3786              5.41     Medium          Low            56
5              732              3.20       High         High            42
6             5400              4.69       High                         56
  wildfire_incidents rainfall_change_mm air_quality_index climate_risk_score
1                 25               -117               127                 89
2                197                 NA               103                 81
3                161                127               253                 71
4                191                -57               166                 57
5                 76                -22               142                 81
6                 89                -24                75                 30
  population_affected_m year
1                  92.7 2026
2                  95.4 2026
3                  32.0 2026
4                 131.5 2026
5                  93.4 2026
6                  88.7 2026
> dim(data)
[1] 5000   16
```

The read.csv() command loads dataset into R. head(data) shows the first 6 rows and dim(data) shows the dataset has 5,000 rows and 16 columns.

Handel Missing Value:

Firstly replace blank cells with ‘NA’. Then count missing values per column. Lastly handel the missing value by fiilling numeric columns with mean and categorical columns with mode.

```
data[data == ""] <- NA
data[data == "NA"] <- NA

sapply(data, function(x) sum(is.na(x)))


num_cols <- sapply(data, is.numeric)
for(col in names(data)[num_cols]) {
  data[[col]][is.na(data[[col]])] <- mean(data[[col]], na.rm = TRUE)
}


get_mode <- function(x) {
  ux <- na.omit(unique(x))
  ux[which.max(tabulate(match(x, ux)))]
}

data$flood_risk[is.na(data$flood_risk)]     <- get_mode(data$flood_risk)
data$drought_risk[is.na(data$drought_risk)] <- get_mode(data$drought_risk)
```

```
> sapply(data, function(x) sum(is.na(x)))
            record_id               country                region     avg_temperature_c
                    0                     0                     0                     2
 temperature_change_c      co2_emissions_mt     sea_level_rise_mm            flood_risk
                    0                     0                     0                     9
         drought_risk         heatwave_days    wildfire_incidents    rainfall_change_mm
                    6                     5                     4                     2
    air_quality_index    climate_risk_score population_affected_m                  year
                    2                     3                     4                     0
>
```

```
> sapply(data, function(x) sum(is.na(x)))
            record_id               country                region     avg_temperature_c
                    0                     0                     0                     0
 temperature_change_c      co2_emissions_mt     sea_level_rise_mm            flood_risk
                    0                     0                     0                     0
         drought_risk         heatwave_days    wildfire_incidents    rainfall_change_mm
                    0                     0                     0                     0
    air_quality_index    climate_risk_score population_affected_m                  year
                    0                     0                     0                     0
>
```

First output there are lots of missing values both in catagorical data and numeric data. So for catagorical data, missing values recoverd by using mode and for numeric data, missing values recoverd by average. Lastly in second output, there are no missing value left.

Detect & Handle Outliers:

Outliers are extreme values that deviate significantly from other observations in a dataset. Remove outliers using IQR method. Using the formula IQR = Q3 - Q1.

```
Q1 <- quantile(data$wildfire_incidents, 0.25, na.rm = TRUE)
Q3 <- quantile(data$wildfire_incidents, 0.75, na.rm = TRUE)
IQR_val <- Q3 - Q1

data_clean <- data[
  data$wildfire_incidents >= (Q1 - 1.5 * IQR_val) &
    data$wildfire_incidents <= (Q3 + 1.5 * IQR_val), ]
```

```
>
> # Compare before and after
> nrow(data)        # before
[1] 5000
> nrow(data_clean)  # after
[1] 4997
>
```

The code removed 3 outlier rows from the wildfire_incidents column. It reducing the dataset from 5000 to 4997 observations while preserving the integrity of the remaining data.

Remove Duplicate Rows:

There are some duplicate rows in the dataset. This need to count rows and remove them.

```
sum(duplicated(data_clean))

data_no_dup <- data_clean[!duplicated(data_clean), ]
```

```
> sum(duplicated(data_clean))
[1] 4
```

There are 4 duplicate rows counted and remove them from dataset.

Handle Invalid Data:

This dataset has some invalid data. It will remove the rows.

```
sum(data_filtered$co2_emissions_mt < 0)
sum(data_filtered$heatwave_days < 0)
sum(data_filtered$wildfire_incidents < 0)
sum(data_filtered$air_quality_index < 0)
sum(data_filtered$population_affected_m < 0)

data_valid <- data_filtered[
  data_filtered$co2_emissions_mt >= 0 &
    data_filtered$heatwave_days >= 0 &
    data_filtered$wildfire_incidents >= 0 &
    data_filtered$air_quality_index >= 0 &
    data_filtered$population_affected_m >= 0, ]
```

```
> # Check for negative values in columns that must be positive
> sum(data_filtered$co2_emissions_mt < 0)
[1] 0
> sum(data_filtered$heatwave_days < 0)
[1] 0
> sum(data_filtered$wildfire_incidents < 0)
[1] 0
> sum(data_filtered$air_quality_index < 0)
[1] 0
> sum(data_filtered$population_affected_m < 0)
[1] 0
```

No data cleaning was required because all values in these columns are already valid. The dataset remains unchanged with all rows retained.

Convert Data Types:

Categorical variables contain text based labels "High", "Medium", "Low" that need to be converted to factor type for proper statistical analysis in R Convert categorical columns to numaric.

```
data_clean$country <- as.factor(data_clean$country)
data_clean$region <- as.factor(data_clean$region)
data_clean$flood_risk <- as.factor(data_clean$flood_risk)
data_clean$drought_risk <- as.factor(data_clean$drought_risk)
```

```
table(data_clean$flood_risk)
table(data_clean$drought_risk)
str(data_clean)
```

```
> str(data_clean)
'data.frame':   4997 obs. of  16 variables:
 $ record_id            : chr  "CLM0001" "CLM0002" "CLM0003" "CLM0004" ...
 $ country              : Factor w/ 10 levels "Australia","Brazil",..: 3 2 1 5 10 9 10 3 7 3 ...
 $ region               : Factor w/ 6 levels "Africa","Asia",..: 4 6 5 3 4 3 4 4 1 4 ...
 $ avg_temperature_c    : num  16.8 28.8 6.6 10.9 13.9 6.2 24.9 22.4 22.4 17.1 ...
 $ temperature_change_c : num  2.4 1.51 1.7 1.53 2.07 0.64 1.49 1.87 1.33 1.51 ...
 $ co2_emissions_mt     : num  5477 3758 1904 3786 732 ...
 $ sea_level_rise_mm    : num  1.44 1.55 5.81 5.41 3.2 4.69 4.49 2.78 4.61 1.41 ...
 $ flood_risk           : Factor w/ 3 levels "High","Low","Medium": 2 3 3 3 1 1 2 1 2 1 ...
 $ drought_risk         : Factor w/ 3 levels "High","Low","Medium": 1 3 3 2 1 3 3 3 2 3 ...
 $ heatwave_days        : num  9 38 20 56 42 56 40 24 19 24 ...
 $ wildfire_incidents   : num  25 197 161 191 76 89 113 178 46 141 ...
 $ rainfall_change_mm   : num  -117 0.894 127 -57 -22 ...
 $ air_quality_index    : num  127 103 253 166 142 75 153 230 65 74 ...
 $ climate_risk_score   : num  89 81 71 57 81 ...
 $ population_affected_m: num  92.7 95.4 32 131.5 93.4 ...
 $ year                 : num  2026 2026 2026 2026 2026 ...
> table(data_clean$flood_risk)

  High    Low Medium
  1596   1741   1660
> table(data_clean$drought_risk)

  High    Low Medium
  1654   1622   1721
```

After conversion, flood_risk has 1596 High, 1741 Low and 1660 Medium values then drought_risk shows 1654 High, 1622 Low and 1721 Medium values. Now the dataset contains 4997 observations.

Normalization:

Normalization convert numerical data to a range (0 to 1) using the formula:
(x - min) / (max - min). Apply it to continuous columns.

```
min_max_norm <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

data_clean$avg_temperature_c_norm  <- min_max_norm(data_clean$avg_temperature_c)
data_clean$co2_emissions_mt_norm   <- min_max_norm(data_clean$co2_emissions_mt)
data_clean$climate_risk_score_norm <- min_max_norm(data_clean$climate_risk_score)

summary(data_clean$avg_temperature_c_norm)
summary(data_clean$co2_emissions_mt_norm)
summary(data_clean$climate_risk_score_norm)
```

```
> summary(data_clean$avg_temperature_c_norm)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.0000  0.2520  0.4960  0.4999  0.7520  1.0000
> summary(data_clean$co2_emissions_mt_norm)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.0000  0.2475  0.5047  0.4996  0.7468  1.0000
> summary(data_clean$climate_risk_score_norm)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.00000 0.05000 0.10000 0.09919 0.15000 1.00000
>
```

All 3 normalized columns have minimum value = 0 and maximum value = 1. All values has between 0 and 1.

Filter Data:

In the flood risk column there are 3 outputs low, medium and high. So for filteration it keep only high and medium flood risk.

```
table(data_no_dup$flood_risk)

data_filtered <- data_no_dup[data_no_dup$flood_risk %in% c("High", "Medium"), ]

table(data_filtered$flood_risk)
nrow(data_filtered)
```

```
> table(data_no_dup$flood_risk)

  High    Low Medium
  1594   1739   1660
> table(data_filtered$flood_risk)

  High    Low Medium
  1594      0   1660
> nrow(data_filtered)
[1] 3254
>
```

After filtering, the dataset contains 1594 High and 1660 Medium risk observations, totaling 3254 rows. All 1739 Low risk rows have been removed from the dataset.

Balance the Dataset:

Balancing ensures each categorical variable has an equal number of observations and it preventing bias in machine learning models.

```
library(dplyr)

data_valid$flood_risk <- droplevels(data_valid$flood_risk)

table(data_valid$flood_risk)

min_size <- min(table(data_valid$flood_risk))
min_size

set.seed(42)
data_balanced <- data_valid %>%
  group_by(flood_risk) %>%
  sample_n(min_size) %>%
  ungroup()
```


```
library(dplyr)

data_valid$flood_risk <- droplevels(data_valid$flood_risk)

table(data_valid$flood_risk)

min_size <- min(table(data_valid$flood_risk))
min_size

set.seed(42)
data_balanced <- data_valid %>%
  group_by(flood_risk) %>%
  sample_n(min_size) %>%
  ungroup()
```


After balancing, all three flood risk categories (High, Low, Medium) will have exactly 1594 observations each, creating a perfectly balanced dataset with equal.

Train/Test Split (70/30):

Splitting data into training and testing sets is essential for evaluating machine learning models. The training set builds the model and the testing set validates its performance.

```
n <- nrow(data_balanced)
n

set.seed(123)
train_index <- sample(1:n, size = 0.7 * n)

train_data <- data_balanced[train_index, ]
test_data  <- data_balanced[-train_index, ]

nrow(train_data)
nrow(test_data)
```

```
> n <- nrow(data_balanced)
> n
[1] 3188
>
> set.seed(123)
> train_index <- sample(1:n, size = 0.7 * n)
>
> train_data <- data_balanced[train_index, ]
> test_data  <- data_balanced[-train_index, ]
>
> nrow(train_data)  # 70%
[1] 2231
> nrow(test_data)   # 30%
[1] 957
>
```

The dataset is successfully split into 2231 training observations (70%) and 957 testing observations (30%), maintaining the original data distribution across both subsets.

Descriptive Statistics by Target Class:

Descriptive statistics summarize and compare the distribution of numeric variables across different categories of a target variable.

```
tapply(data_balanced$avg_temperature_c, data_balanced$flood_risk, summary)
tapply(data_balanced$co2_emissions_mt, data_balanced$flood_risk, summary)
tapply(data_balanced$climate_risk_score, data_balanced$flood_risk, summary)
tapply(data_balanced$heatwave_days, data_balanced$flood_risk, summary)
```

```
> tapply(data_balanced$avg_temperature_c, data_balanced$flood_risk, summary)
$High
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    5.0    11.4    17.6    17.6    24.0    30.0

$Medium
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   11.30   17.30   17.59   24.00   29.90

> tapply(data_balanced$co2_emissions_mt, data_balanced$flood_risk, summary)
$High
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    302    1644    3068    3097    4537    5999

$Medium
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    308    1797    3248    3201    4574    5998

> tapply(data_balanced$climate_risk_score, data_balanced$flood_risk, summary)
$High
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  20.00   40.00   58.00   57.93   77.00   95.00

$Medium
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  20.00   39.00   59.00   58.34   78.00   95.00

> tapply(data_balanced$heatwave_days, data_balanced$flood_risk, summary)
$High
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   5.00   18.00   31.50   32.07   46.00   60.00
```

The summary outputs show that High and Medium flood risk categories have remarkably similar statistical profiles.

Compare Average Values Across Two Categories:

This analysis calculates the mean (average) of key numeric variables for each drought risk category (High, Low, Medium).

```
tapply(data_balanced$avg_temperature_c, data_balanced$drought_risk, mean, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$drought_risk, mean, na.rm = TRUE)
tapply(data_balanced$climate_risk_score, data_balanced$drought_risk, mean, na.rm = TRUE)
```

```
> tapply(data_balanced$avg_temperature_c, data_balanced$drought_risk, mean, na.rm = TRUE)
    High      Low   Medium
17.68168 17.57191 17.52871
> tapply(data_balanced$co2_emissions_mt, data_balanced$drought_risk, mean, na.rm = TRUE)
    High      Low   Medium
3171.213 3116.995 3158.886
> tapply(data_balanced$climate_risk_score, data_balanced$drought_risk, mean, na.rm = TRUE)
    High      Low   Medium
57.65206 59.03861 57.74638
>
```

Compare Variability Across Categories:

Variability metrics (Standard Deviation, Variance, IQR, and Range) measure how spread out CO2 emissions values are within each region.

```
tapply(data_balanced$co2_emissions_mt, data_balanced$region, sd, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region, var, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region, IQR, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region,
       function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
```

```
> # SD, Variance, IQR and Range of co2_emissions_mt across regions
> tapply(data_balanced$co2_emissions_mt, data_balanced$region, sd, na.rm = TRUE)
       Africa          Asia        Europe North America       Oceania South America
     1552.605      1632.878      1645.694      1640.734      1692.238      1601.808
> tapply(data_balanced$co2_emissions_mt, data_balanced$region, var, na.rm = TRUE)
       Africa          Asia        Europe North America       Oceania South America
      2410582       2666290       2708310       2692009       2863668       2565789
> tapply(data_balanced$co2_emissions_mt, data_balanced$region, IQR, na.rm = TRUE)
       Africa          Asia        Europe North America       Oceania South America
      2689.75       2875.00       2884.50       2901.50       2813.00       2713.75
> tapply(data_balanced$co2_emissions_mt, data_balanced$region,
+        function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
       Africa          Asia        Europe North America       Oceania South America
         5592          5688          5689          5684          5695          5658
>
```

Project Code

```r
setwd("D:/Fall 25-26/Data Science/Project")

list.files()

data <- read.csv("data_science_project.csv", stringsAsFactors = FALSE)

head(data)
dim(data)


data[data == ""] <- NA
data[data == "NA"] <- NA

sapply(data, function(x) sum(is.na(x)))

num_cols <- sapply(data, is.numeric)

for(col in names(data)[num_cols]) {
  data[[col]][is.na(data[[col]])] <- mean(data[[col]], na.rm = TRUE)
}

get_mode <- function(x) {
  ux <- na.omit(unique(x))
  ux[which.max(tabulate(match(x, ux)))]
}

data$flood_risk[is.na(data$flood_risk)]     <- get_mode(data$flood_risk)
data$drought_risk[is.na(data$drought_risk)] <- get_mode(data$drought_risk)

data$wildfire_incidents <- as.numeric(data$wildfire_incidents)
data$air_quality_index  <- as.numeric(data$air_quality_index)
data$climate_risk_score <- as.numeric(data$climate_risk_score)

data$wildfire_incidents[is.na(data$wildfire_incidents)] <-
  mean(data$wildfire_incidents, na.rm = TRUE)

data$air_quality_index[is.na(data$air_quality_index)] <-
  mean(data$air_quality_index, na.rm = TRUE)

data$climate_risk_score[is.na(data$climate_risk_score)] <-
  mean(data$climate_risk_score, na.rm = TRUE)

sapply(data, function(x) sum(is.na(x)))
```

```r
boxplot(data$wildfire_incidents,
        main = "Outliers in Wildfire Incidents",
        col = "lightblue",
        ylab = "Wildfire Incidents")

Q1 <- quantile(data$wildfire_incidents, 0.25, na.rm = TRUE)
Q3 <- quantile(data$wildfire_incidents, 0.75, na.rm = TRUE)
IQR_val <- Q3 - Q1

data_clean <- data[
  data$wildfire_incidents >= (Q1 - 1.5 * IQR_val) &
    data$wildfire_incidents <= (Q3 + 1.5 * IQR_val), ]

nrow(data)
nrow(data_clean)
```

```r
data_clean$country      <- as.factor(data_clean$country)
data_clean$region       <- as.factor(data_clean$region)
data_clean$flood_risk   <- as.factor(data_clean$flood_risk)
data_clean$drought_risk <- as.factor(data_clean$drought_risk)

table(data_clean$flood_risk)

get_mode <- function(x) {
  ux <- na.omit(unique(x))
  ux[which.max(tabulate(match(x, ux)))]
}

data_clean$flood_risk <- as.character(data_clean$flood_risk)

data_clean$flood_risk[!data_clean$flood_risk %in% c("High", "Medium", "Low")] <-

data_clean$flood_risk <- as.factor(data_clean$flood_risk)

str(data_clean)
table(data_clean$flood_risk)
table(data_clean$drought_risk)
```

```r
min_max_norm <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

data_clean$avg_temperature_c_norm  <- min_max_norm(data_clean$avg_temperature_c)
data_clean$co2_emissions_mt_norm   <- min_max_norm(data_clean$co2_emissions_mt)
data_clean$climate_risk_score_norm <- min_max_norm(data_clean$climate_risk_score)

summary(data_clean$avg_temperature_c_norm)
summary(data_clean$co2_emissions_mt_norm)
summary(data_clean$climate_risk_score_norm)


sum(duplicated(data_clean))

data_no_dup <- data_clean[!duplicated(data_clean), ]

nrow(data_clean)
nrow(data_no_dup)

table(data_no_dup$flood_risk)

data_filtered <- data_no_dup[data_no_dup$flood_risk %in% c("High", "Medium"), ]

table(data_filtered$flood_risk)
nrow(data_filtered)

sum(data_filtered$co2_emissions_mt < 0)
sum(data_filtered$heatwave_days < 0)
sum(data_filtered$wildfire_incidents < 0)
sum(data_filtered$air_quality_index < 0)
sum(data_filtered$population_affected_m < 0)

data_valid <- data_filtered[
  data_filtered$co2_emissions_mt >= 0 &
    data_filtered$heatwave_days >= 0 &
    data_filtered$wildfire_incidents >= 0 &
    data_filtered$air_quality_index >= 0 &
    data_filtered$population_affected_m >= 0, ]

nrow(data_filtered)
nrow(data_valid)
```

```r
library(dplyr)

data_valid$flood_risk <- droplevels(data_valid$flood_risk)

table(data_valid$flood_risk)

min_size <- min(table(data_valid$flood_risk))
min_size

set.seed(42)
data_balanced <- data_valid %>%
  group_by(flood_risk) %>%
  sample_n(min_size) %>%
  ungroup()

table(data_balanced$flood_risk)
nrow(data_balanced)



set.seed(123)
train_index <- sample(1:n, size = 0.7 * n)

train_data <- data_balanced[train_index, ]
test_data  <- data_balanced[-train_index, ]

nrow(train_data)
nrow(test_data)


tapply(data_balanced$avg_temperature_c, data_balanced$flood_risk, summary)
tapply(data_balanced$co2_emissions_mt, data_balanced$flood_risk, summary)
tapply(data_balanced$climate_risk_score, data_balanced$flood_risk, summary)
tapply(data_balanced$heatwave_days, data_balanced$flood_risk, summary)


tapply(data_balanced$avg_temperature_c, data_balanced$drought_risk, mean, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$drought_risk, mean, na.rm = TRUE)
tapply(data_balanced$climate_risk_score, data_balanced$drought_risk, mean, na.rm = TRUE)


tapply(data_balanced$co2_emissions_mt, data_balanced$region, sd, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region, var, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region, IQR, na.rm = TRUE)
tapply(data_balanced$co2_emissions_mt, data_balanced$region,
       function(x) max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
```